

Task 07 - Part 01: Question Answers

C# Fundamentals - Constructors, Overloading, Inheritance & Interfaces

Q1. Why does defining a custom constructor suppress the default constructor in C#?

C# only generates an implicit parameterless constructor automatically **when a class defines no constructors at all**. As soon as you write even one custom constructor - with or without parameters - the compiler assumes you're taking full responsibility for how the class is initialized, so it stops auto-generating the default one. This is intentional: if a class requires certain data to be provided at creation time (e.g., an Id), silently allowing a parameterless Car() to also exist could let objects be created in an incomplete or invalid state. If a parameterless constructor is still needed alongside custom ones, it must be written explicitly.

Q2. How does method overloading improve code readability and reusability?

Method overloading lets multiple methods share the **same name** while differing in their parameter lists (number, type, or order of parameters), letting the caller invoke conceptually the same operation - like Sum() - regardless of exactly what data they're working with. This improves readability because callers don't need to remember different names for essentially the same action (e.g., SumTwoInts, SumThreeInts, SumDoubles), and it improves reusability because the class's public API stays clean and consistent while still supporting a variety of input scenarios. The compiler resolves the correct overload to call based on the arguments provided at compile time.

Q3. What is the purpose of constructor chaining in inheritance?

Constructor chaining (using the **base(...)** keyword in a derived class's constructor) ensures that a base class's own initialization logic runs and properly sets up its fields/properties **before** the derived class's constructor adds its own initialization on top. This avoids duplicating the base class's setup code inside every derived class, keeps initialization logic centralized in one place (the base constructor), and guarantees that an object is never left with an improperly initialized base part, since a derived class's constructor always implicitly or explicitly calls a base constructor before its own body executes.

Q4. How does new differ from override in method overriding?

override provides true (runtime) polymorphism: it replaces the base class's implementation for that method, and which version runs is determined by the **actual runtime type** of the object, even when accessed through a base-class reference or variable - this requires the base method to be marked virtual (or abstract). **new** instead **hides** the base class's method with an unrelated new one of the same name/signature; there is no polymorphic relationship between them. With new, which method runs is determined by the **compile-time type of the reference** used to call it - calling the method through a Parent-typed reference to a Child object will run the Parent version, while calling it through a Child-typed reference runs the Child version, even for the same underlying object.

Q5. Why is ToString() often overridden in custom classes?

By default, ToString() (inherited from System.Object) just returns the object's fully-qualified type name, which isn't meaningful for understanding an instance's actual data. Overriding it lets a class control exactly how its instances are represented as text - which is especially valuable for debugging, logging, and simple output, since Console.WriteLine(obj) and string interpolation both call ToString() automatically. It's also a natural place to demonstrate polymorphism: overriding ToString() differently in a base class and a derived class means the correct, type-appropriate text representation is produced automatically, even when printing through a base-class reference.

Q6. Why can't you create an instance of an interface directly?

An interface only defines a **contract** - a set of members (methods, properties, events) that any implementing type must provide - but it contains no actual field storage or (traditionally) no executable implementation of its own. Since an interface has no concrete data or complete behavior of its own, there's nothing meaningful to construct - the runtime needs a real, concrete type to know how much memory to allocate and what code to actually run for each member. You can, however, declare a variable of the interface type and assign it an **instance of a class that implements that interface**, which is how interfaces are actually used in practice.

Q7. What are the benefits of default implementations in interfaces introduced in C# 8.0?

Default interface methods let an interface provide a **ready-made implementation** for a member, which implementing classes automatically get for free unless they choose to override it. The main benefit is being able to **add new members to an existing interface without breaking every class that already implements it** - without default implementations, adding a new method to an interface would force every implementing class to be updated immediately or fail to compile. This makes interfaces more flexible for API evolution over time, while still letting individual implementing classes override the default behavior whenever they need custom logic instead.

Q8. Why is it useful to use an interface reference to access implementing class methods?

Using an interface-typed reference (e.g., IMovable car = new Car();) lets code depend only on the **contract** (what the object can do) rather than on a specific concrete class (how it does it). This supports loose coupling and polymorphism: the same code that calls Move() through an IMovable reference can work with a Car, a Bicycle, or any other class implementing IMovable, without needing to know or care about the concrete type. It makes code more flexible, easier to extend with new implementations later, and much easier to test (e.g., swapping in a mock implementation of the interface).

Q9. How does C# overcome the limitation of single inheritance with interfaces?

A C# class can only inherit from **one** base class, but it can implement **any number of interfaces** at the same time. Since interfaces just define contracts (what a type must be able to do) rather than full

implementation and state the way base classes do, allowing multiple interface implementations avoids the ambiguity problems that multiple class inheritance can cause (such as conflicting inherited state or the "diamond problem"), while still letting a single class combine several unrelated sets of behavior/capabilities - for example, a File class can implement both IReadable and IWritable simultaneously, effectively achieving many of the benefits of multiple inheritance in a safer, more structured way.

Q10. What is the difference between a virtual method and an abstract method in C#?

A **virtual** method is declared in a base class **with a full, default implementation** that derived classes are free to override if they choose to, but are not required to - if a derived class doesn't override it, the base implementation runs as-is. An **abstract** method has **no implementation at all** in the base class - it only declares the method's signature - and it can only exist inside an abstract class. Every non-abstract derived class is **required** to override and provide an implementation for every abstract method it inherits, or the code won't compile. In short: virtual means "here's a default, override it if you want," while abstract means "there is no default, every concrete subclass must supply its own implementation."